

Improving Deep Learning Based Second-Order Side-Channel Analysis With Bilinear CNN

Pei Cao^{ID}, Chi Zhang^{ID}, Xiangjun Lu, Dawu Gu^{ID}, and Sen Xu

Abstract—In recent years, deep learning techniques have received significant attention in the side-channel community due to their state-of-the-art performance in profiled attacks against embedded devices. Compared with template attacks, deep learning-based attacks can deal with the high dimensionality of trace and misalignment without pre-processing. However, the performance of attacks is very sensitive to the network architecture, especially when considering masking countermeasures. Although previous works have shown the potential of neural networks to break the first-order masking, the inner-working of how the network combines the leakage of mask and masked value is still unclear and could be suboptimal. To reduce this gap, we propose to embed product combination, which has been proved to be the best combination function in noisy situations, into the design of neural networks. To this end, we introduce a bilinear convolutional neural network (in short, B-CNN) for efficient profiled attacks against the widely used masking countermeasure. In order to interpret the inner-working and decision-making of B-CNN, we propose a new visualization tool called layer-wise correlation that can reveal the points of interest and help to understand the combination of leakages. We evaluate our networks on several public datasets, e.g., ASCAD and CHES CTF 2018. The results indicate that B-CNN converges significantly faster than classic CNN models, even using a very limited number of profiling traces (e.g., 8000 profiling traces for the ASCAD dataset). Moreover, our networks perform even systematically better (w.r.t. the number of attack traces) than the current state-of-the-art results on all the investigated datasets.

Index Terms—Side-channel attacks, profiled attacks, deep learning, masking countermeasures.

I. INTRODUCTION

SIDE-CHANNEL analysis is a realistic threat to modern implementations of cryptosystems in embedded devices. It exploits the vulnerability of a system by analyzing its physical properties (e.g., power consumption [1] and electromagnetic emissions [2]) to recover some leakage about its secrets. As a branch of SCA, the profiled attack is considered as one

of the most powerful SCAs, at least from the information theory point of view [3]. In such a scenario, the attacker gets access to a profiling device prior to the attack. The profiling device is open to the attacker that can be used to estimate the conditional distribution associated with sensitive variables. Then he can predict the right sensitive value of the target device to retrieve its secret by using multiple side-channel traces. In 2002, template attack (TA) [3], which is considered the first profiled attack, is proposed. Since the differential analysis in the attack phase of profiled attacks is based on the distinguishment among different classes, following the seminal TA, various machine learning techniques are introduced and adopted into the context of SCA [4], [5], [6], [7].

To distort the distinguishment among classes, a very common countermeasure is to randomize the sensitive variables by masking techniques [8], [9], [10]. For example, in the first-order Boolean masking scheme, every sensitive variable is randomly split into two shares: a mask and a masked value. As a result, there is no first-order leakage on the unmasked sensitive variable. To break the masking countermeasure, the attacker needs to get information on all of the shares and combine them to get information on the sensitive variable, which is called high-order attacks. In previous literature, two combination functions are most widely used in pre-processing: the product of the two leakage values [11] and the absolute value of their difference [12]. The authors in [13] further proved that the centered product combination function is the best possible combination function in noisy situations. However, high-order attacks imply higher costs in terms of the dimension of traces and computational complexity.

Recently, deep learning-based SCA presents a powerful option for profiling SCA. Many results indicate that deep learning techniques could directly cater to masking countermeasures [14], [15], [16], [17], even with the addition of misalignment [15], [18]. In such a scenario, neural networks, e.g., multilayer perceptron (MLP) and convolutional neural network (CNN), are used to automatically combine leakage of the mask and masked value without pre-processing. In such a context, the neural network has to handle two tasks: 1) learning a combination function and 2) learning to identify the sensitive values based on the combined leakage. Due to the addition of task 1, masked implementations are significantly more difficult to attack compared to an unprotected target. Although the methodology for efficient network architecture has been deeply discussed in [17] and [19], when targeting masked implementations, it is still challenging to design a well-performed network architecture that can steadily con-

Manuscript received 4 March 2022; revised 30 June 2022; accepted 11 October 2022. Date of publication 25 October 2022; date of current version 23 November 2022. This work was supported by the National Natural Science Foundation of China under Grant 62072307. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Ulrich Rührmair. (Corresponding author: Dawu Gu.)

Pei Cao and Dawu Gu are with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200200, China, and also with the State Key Laboratory of Cryptology, Beijing 100878, China (e-mail: loccs_cp@sjtu.edu.cn; dwgu@sjtu.edu.cn).

Chi Zhang and Xiangjun Lu are with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200200, China (e-mail: zcsjtu@sjtu.edu.cn; luxiangjun@sjtu.edu.cn).

Sen Xu is with ViewSources Technology Company Ltd., Shanghai 200200, China (e-mail: sen.xu@viewsources.com).

Digital Object Identifier 10.1109/TIFS.2022.3216959

1556-6021 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

verge. Even if the network has converged, it may still learn a suboptimal combination function that results in suboptimal attacking performance. So far, the inner-working of how the network combines the leakages is still unclear. To the best of our knowledge, no literature in this direction has discussed this issue in-depth.

The primary motivation of this work is to design a deep learning architecture for second-order profiled SCA that is easier to converge and has a better attacking performance. For the first time, we propose to explicitly embed the product combination of leakages into the design of neural networks. To this end, we introduce the bilinear convolutional neural network (denoted B-CNN) that contains two CNN streams. Specifically, one stream is used to extract leakage features that are related to the mask, and the other extracts features of the masked value. Then their outputs are combined using the outer product to obtain a bilinear vector, which is used for further predictions. Intuitively, B-CNN simplifies the learning tasks of the neural network (by getting rid of task 1), which makes the network easier to converge than the classic CNN. Further, explicitly leveraging the product combination can prevent the neural network from falling into a locally optimal solution (i.e., learn a suboptimal combination function). To evaluate our methodology, we further propose a novel visualization tool called layer-wise correlation, which can help to understand the inner-working and decision-making of B-CNN.

A. Related Works

Our work is related to a series of papers that utilize deep learning to target masked implementations. Maghrebi et al. [14] first adopted deep learning to profiled SCA against the masking countermeasure. The authors argue that the non-linear activation function (e.g., a sigmoid) implicitly performs combinations of the mask and masked value exactly like a second-order side-channel attack. Prouff et al. [15] introduced the first open dataset with masking countermeasure (named ASCAD) that serves as a common basis for further works on the application of deep learning in the SCA context. Based on public datasets, Zaid et al. [17] provided the first step towards a systematic approach for building suitable architecture, which outperforms Prouff's model while reducing network complexity. At the end of [17], Zaid et al. also provided a brief proof showing that neural networks could combine the mask and masked value following the absolute difference combining function. Wouters et al. [19] showed the importance of pre-processing for deep learning based SCA to reduce the network complexity. Won et al. [20] proposed a generic multi-scale CNN framework that integrated side-channel oriented pre-processing into deep learning architecture to boost its performance. Rioja et al. [21] proposed a battery of automated (Estimation of Distribution Algorithm-based) profiled attacks that aimed at minimizing human intervention in fine-tuning the model. Perin et al. [22] showed ensembles of deep learning models could further improve the performance of profiled SCA. Rijdsdijk et al. [23] highlighted the importance of network hyperparameters and proposed the reinforcement learning framework for hyperparameter tuning. Scanning through the series of previous works, we notice that

the majority of the research adopts classic neural networks, such as MLP and CNN. To the best of our knowledge, no work has investigated the possibility of strengthening deep learning based SCA with the capability of integrating existing high-order combination techniques.

Our work is also related to papers focusing on bilinear pooling, which is widely used for fine-grained visual recognition [24] and multimodal fusion [25]. As far as we know, bilinear neural networks have not been explored in the side-channel community to deal with the masking countermeasure.

B. Our Contributions

In this paper, we take one step further and bridge product combination with deep learning based profiled attacks. The main contributions of this work can be summarized as follows:

- **An architecture for efficient second-order profiled attacks.** We propose a new architecture (called B-CNN) in the context of profiled SCA, which embeds the product combination of leakage into neural networks. The motivation is to simplify the learning tasks of the network and prevent it from falling into locally optimal solutions by explicitly introducing the product combination function. Results show that B-CNN converges faster and reaches significantly better attack performance compared with classic CNN models.
- **A methodology to further interpret and understand the inner-working and decision-making of B-CNN.** We propose layer-wise correlation (in short, LWC) for interpreting the performance of our network. We show that LWC can not only visualize how it selects its features (i.e., points of interest related to the mask and masked value), but also can reveal whether and where the network combines the leakages, which could help to build efficient architectures in terms of attack efficiency and network complexity.
- **State-of-the-art results on public datasets.** We evaluate our architecture on popular public datasets (ASCAD and CHES CTF 2018), which have masking countermeasures. We show that using the same amount of profiling traces, our approach significantly outperforms the current state-of-the-art results. Further, we show that B-CNN can successfully converge using a very limited number of profiling traces (e.g., 8000 traces in the ASCAD dataset). The same amount of profiling traces cannot lead to a converged model using classic CNN proposed by the previous work. Besides, to the best of our knowledge, we are the first to report successful profiled attacks using the identity leakage model on the CHES CTF 2018 dataset. All these experiments can be reproduced through the following Github repository: <https://github.com/locos-cp/Bilinear-CNN-for-SCA>.

C. Paper Organization

The remainder of this paper is organized as follows. In Section II, we provide some basics of profiled SCA, high-order attacks, and necessary background about neural networks. In Section III, we present the proposed B-CNN and a

new visualization tool applied to the SCA context. Section IV presents a set of public datasets and experimental results to evaluate our claims. In Section V, we discuss the application of our approach in higher-order masking and other masking schemes. Finally, conclusions are drawn in Section VI.

II. PRELIMINARIES

A. Notation

Throughout this paper we use calligraphic letters as $\mathcal{X}$ to denote sets, the corresponding capital letter X to denote random variables (resp. random vectors $\mathbf{X}$), and the lowercase x (resp. $\mathbf{x}$) to denote their realizations. The probability space of a set $\mathcal{X}$ is denoted by $\mathcal{P}(\mathcal{X})$. If $\mathcal{X}$ is discrete, $\mathcal{P}(\mathcal{X})$ corresponds to the set of vectors $[0, 1]^{|\mathcal{X}|}$ such that the coordinates sum to 1. Let $\Pr[X]$ denote the distribution of X and $\Pr[X = x]$ the probability when X equals x . For profiled attacks, the target sensitive variable is $V = f(P, K)$ where f denotes a cryptographic primitive (e.g., the SubBytes operation), P denotes a known variable (e.g., plaintext or ciphertext) and $K (\in \mathcal{K})$ denotes a part of the secret key (e.g., a byte) that an attacker tries to recover. Among all the possible values K may take, k^* will denote the right key hypothesis. Side-channel traces will be viewed as discrete realizations of a random vector $\mathbf{x} = x_1, \dots, x_l$, with l being the number of features. All N_p profiling traces can be stacked together in a matrix D as follows:

$$D = \begin{pmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_l^{(1)} \\ x_1^{(2)} & \ddots & \ddots & x_l^{(2)} \\ \vdots & \ddots & \ddots & \vdots \\ x_1^{(N_p)} & x_2^{(N_p)} & \dots & x_l^{(N_p)} \end{pmatrix}. \quad (1)$$

We use y to denote the label of a trace, which can be performed using the value or Hamming weight (HW) of the sensitive variable V .

B. Profiled Side-Channel Attacks

Profiled SCA considers a strong attacker that has access to a clone device prior to the attack. In such a scenario, profiled SCA operates in two phases: a profiling/training phase and an attacking/testing phase. In the profiling phase, the attacker acquires N_p side-channel traces $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N_p}$ with known labels (calculated using known plaintext/ciphertext and known key). The goal of the profiling phase is to estimate a probability density function (e.g., $\Pr[\mathbf{X}|Y = y]$ in TA) that could be used to predict the label of attack traces. In other words, the attacker eventually obtain a model $F(\cdot) : \mathcal{X} \rightarrow \mathcal{P}(\mathcal{Y})$. Using the model $F(\cdot)$, the attacker can compute the likelihood for each possible y for a given unseen attack trace. In the attacking phase, the attacker then acquires N_a side-channel traces $\{\mathbf{x}_i\}_{i=1}^{N_a}$ from the target device with a fixed unknown key k^* . To recover k^* and prevent the numeric issue of underflow, he can calculate the log-likelihood score over the attacking traces for each $k \in \mathcal{K}$:

$$\mathbf{d}[k] = \sum_{i=1}^{N_a} \log(F(\mathbf{x}_i)[f(p_i, k)]). \quad (2)$$

Finally, the attacker selects a subkey k_{guess} leading to the highest log-likelihood score: $k_{guess} = \arg\max_{k \in \mathcal{K}} \mathbf{d}[k]$. The attack is successful if $k_{guess} = k^*$.

C. High-Order Attacks

Randomizing the sensitive variables by masking is a very common countermeasure to protect block ciphers implementations. For example, a $(d - 1)$ th-order Boolean masking applied on the AES Sbox can be represented as a series of XOR operations among d shares:

$$v = Sbox(p \oplus k) = v' \oplus m_1 \oplus \dots \oplus m_{d-1}, \quad (3)$$

where $m_1, \dots, m_{d-1}$ are the masks and v' is the masked value. Since the design of a higher order masking scheme that is efficient and secure for $d > 2$ is still an issue, first-order masking is the most widely used to protect block ciphers implementations against DPA, i.e., $v = v' \oplus m$. When first-order masking is applied, the elements m and v' are manipulated at different times t_1 and t_2 and the attacker needs to get information on both of them to get information on v .

The fundamental idea of high-order attacks is to recombine the leakages of mask $L(t_1)$ and masked value $L(t_2)$. In previous literature, three combining functions have been proposed to mount sound second-order attacks against masking implementations:

- 1) Product Combination [11], [26].

$$\mathcal{C}_{prod}(L(t_1), L(t_2)) = L(t_1) \times L(t_2). \quad (4)$$

- 2) Absolute Difference Combination [12].

$$\mathcal{C}_{diff}(L(t_1), L(t_2)) = |L(t_1) - L(t_2)|. \quad (5)$$

- 3) Sine-Based Combination [27].

$$\mathcal{C}_{sin}(L(t_1), L(t_2)) = \sin((L(t_1) - L(t_2))^2). \quad (6)$$

The authors in [13] further proved that the centered product combination function is the best possible combination function in noisy situations.

However, with long side-channel traces, the d shares of $(d - 1)$ th-order masking make the combination very complicated. In the worst case, the attacker has to pre-compute all of the possible combinations of d time samples, and thus yield new traces of length $\binom{l}{d}$. Therefore, masking is still an effective countermeasure in practice that increases the difficulty of carrying out the attacks.

D. Neural Networks in Side-Channel Attacks

Since profiled SCA can be formulated as a *classification problem*, the embedding of deep learning techniques into profiled attacks is straightforward. Specifically, the profiling phase corresponds to the process of training the neural network, while the attacking phase can be regarded as a series of classification tasks. In the profiling phase, a set of parameters θ , named trainable parameters of networks, are updated to generate a classification model $F_\theta(\cdot) : \mathcal{X} \rightarrow \mathcal{P}(\mathcal{Y})$. Then, given an input trace, $F_\theta(\cdot)$ computes an output called a *prediction*. We remark that the *prediction* does not only provide

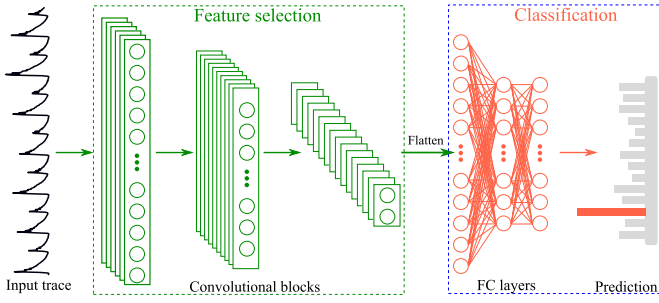


Fig. 1. Classic CNN architecture for side-channel attacks.

the most likely label to solve the classification task, but also the likelihood of all the other labels. In the attacking phase, this form of output allows the attacker to rank the key candidates using multiple attacking traces following Equation 2.

Note that there are many kinds of neural networks, e.g., MLP, CNN, and recurrent neural network (RNN). Their different characteristics are mainly affected by how the neurons are connected within and between layers. In this paper, we focus on the family of CNNs because of their potential of dealing with trace misalignment. A classic architecture of CNN used in previous works is shown in Fig. 1. In this paper, we consider simplifying the structure of CNN by decomposing it into two parts: a feature selection part and a classification part. The feature selection part is mainly composed of several convolutional blocks that aim to extract information related to the target sensitive value to help the classification. The classification part consists of several fully-connected (in short, FC) layers that recombine the neurons to obtain a global result depending on the entire input. In recent literature, CNNs are most widely used to break masked implementations in the presence of desynchronization [15], [17], [18].

1) *How Classic CNN Models Combine Leakages of Mask and Masked Value:* Since convolutional layers only extract local features (kernels combine samples only from a small, local area to form an output), the combination of $L(t_1)$ and $L(t_2)$ should be done by the fully-connected layers. Let assume a CNN model whose feature selection part fully extracts the leakage of mask $L(t_1)$ and masked value $L(t_2)$ such that l_1 neurons (resp. l_2) contain information related to $L(t_1)$ (resp. $L(t_2)$) (see Fig. 2). Then,

$$n_i^{[flatten+1]} = \sum_{j=0}^N (w_j \times n_j^{[flatten]}), \quad (7)$$

where $n_i^{[flatten+1]}$ denotes the value of neuron n_i in the layer of $[flatten + 1]$, N is number of neurons in the flatten layer, w_j is the trainable weight. By setting the weights of irrelevant neurons to zero, Equation 7 can be rewritten as:

$$\begin{aligned} n_i^{[flatten+1]} &= \sum_{j=u}^{u+l_1} (w_j \times n_j^{[flatten]}) + \sum_{j=v}^{v+l_2} (w_j \times n_j^{[flatten]}) \\ &= L(t_1) + L(t_2), \end{aligned} \quad (8)$$

where $L(t_1) = \sum_{j=u}^{u+l_1} (w_j \times n_j^{[flatten]})$ and $L(t_2) = \sum_{j=v}^{v+l_2} (w_j \times n_j^{[flatten]})$. Because $(w_j)_{0 \leq j \leq N} \in \mathbb{R}$, $L(t_1)$ and

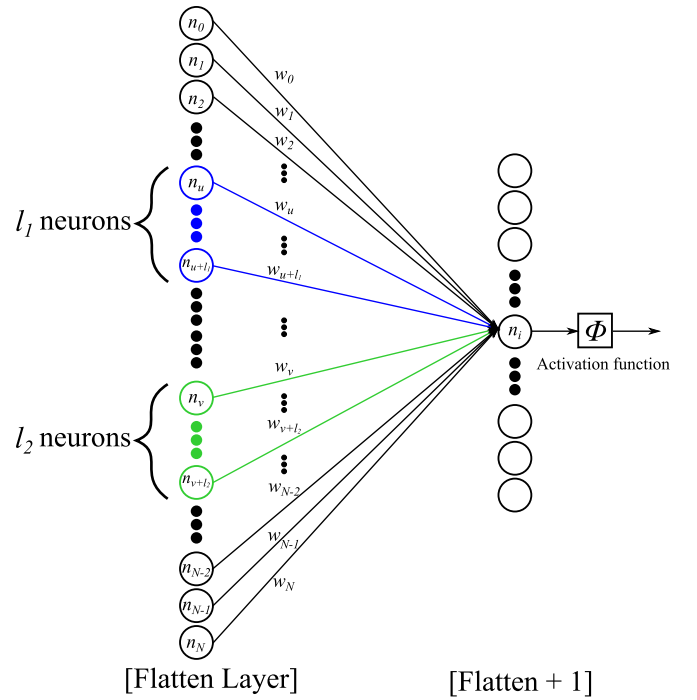


Fig. 2. Combination of the leakage of mask and masked value through fully-connected layers.

$L(t_2)$ can be positive or negative. Therefore, the authors in [17] argued that, if the network is perfectly trained, it can combine the leakages of mask and the masked value following the *absolute difference combining function*. However, we notice that the activation function is not considered in their work, which plays a vital role for a network to learn a non-linear combination function. In this paper, we would argue that the combination function learned by the neural network can be explained according to the *universal approximation theorems* [28]. The theorems imply that neural networks can represent a wide variety of functions when given a nonpolynomial activation function and appropriate weights. Therefore, if the network is perfectly trained, it can learn, theoretically speaking, any of the combination functions mentioned in Section II-C. However, the *universal approximation theorems* typically do not provide a construction for the weights, but merely state that such a construction is possible. Therefore, it is important to find a good network architecture to prevent the network from learning a suboptimal combination function.

2) *How to Evaluate the Performance of Profiled Attack:* To evaluate the performance of profiled SCA, one must select an appropriate metric. While test accuracy and loss are most commonly used in various classification tasks, it has been shown that they are not optimal in the context of profiled SCA [29]. Therefore, we use guessing entropy (GE), a metric commonly used for SCA evaluations, especially in the deep learning context. Typically, GE can be described as the average rank of the correct key after attacks. More specifically, given N_a attack traces, the profiled attack will output an accumulated score vector $\mathbf{d} = [d_1, d_2, \dots, d_{|\mathcal{K}|}]$. Each element of $\mathbf{d}$ denotes the estimated probability of a candidate subkey. By sorting the score vector to get a $\mathbf{d}'$ in decreasing order of score, GE can

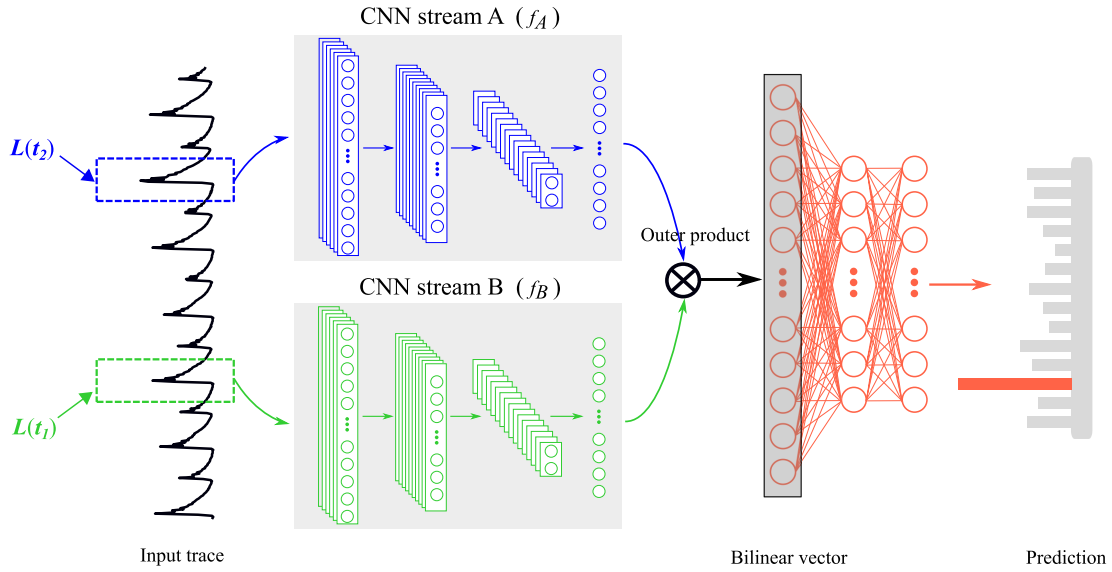


Fig. 3. Main structure for B-CNN.

be defined as the average position of d_{k^*} in $\mathbf{d}'$ over multiple attacks. In the rest of this paper, we use Nt_{GE} to denote the number of attack traces needed to reach a constant GE of 1. To obtain a good estimate of Nt_{GE} , we repeat the attack 100 times with randomly-selected attack traces.

III. BILINEAR CNN FOR DL BASED SCA

Although classic CNN models have shown their potential of automatically combining features in different locations of traces to break masked implementations, we identify two main limitations of the current approaches. For one thing, the two-stage task (combination then classification) makes it difficult for a classic CNN model to converge. For example, we notice that even using state-of-the-art CNN models in [17], training of the network does not always result in a converging model due to the random weight initialization [19]. For another, in noisy situations with limited training data, the network is very likely to learn a suboptimal combination function that leads to inefficient key recovery. Therefore, in this section, we propose the B-CNN which aims to overcome the above two limitations. Subsection III-A describes the bilinear combination and how it is embedded into a common CNN model. Then, in Subsection III-B, we show how we evaluate the feature selection and combination of B-CNN using the proposed layer-wise correlation (LWC). We note that LWC is a general approach that can also be applied in other network architectures.

A. B-CNN Model for SCA

A general formulation of a B-CNN model for profiled SCA consists of a triple $\mathcal{B} = (f_A, f_B, \mathcal{C})$. Herein, f_A and f_B are *feature selection functions*, and $\mathcal{C}$ is a *classification function*. A *feature selection function* is a mapping $f : X \rightarrow \mathbb{R}^{D \times 1}$ that takes an input trace $\mathbf{x}$ and outputs a feature vector of size $D \times 1$. The outputs of f_A and f_B are combined using the matrix outer product, i.e., the bilinear feature combination of f_A and f_B is given by $\text{bilinear}(X, f_A, f_B) = f_A(X)f_B(X)^T$. If f_A

and f_B extract features of size $m \times 1$ and $n \times 1$ respectively, then $\text{bilinear}(X, f_A, f_B)$ is of size $m \times n$. The bilinear vector obtained by reshaping $\text{bilinear}(X, f_A, f_B)$ to size $mn \times 1$ is a general-purpose trace descriptor that can be used with a *classification function* $\mathcal{C}$. Specifically, let us denote M the output of the $\text{bilinear}(X, f_A, f_B)$ such that $M \in \mathbb{R}^{m \times n}$:

$$M = \begin{pmatrix} a_1b_1 & \cdots & a_1b_n \\ \vdots & \ddots & \vdots \\ a_mb_1 & \cdots & a_mb_n \end{pmatrix}, \quad (9)$$

where $\mathbf{a} \in \mathbb{R}^m = f_A(X)$, $\mathbf{b} \in \mathbb{R}^n = f_B(X)$. Then, the input of the $\mathcal{C}$ is a concatenated vector $\mathbf{v}$ that can be constructed:

- Column-wise: $\mathbf{v} = [a_1b_1, a_2b_1, a_3b_1, \dots, a_mb_n]$,
- Row-wise: $\mathbf{v} = [a_1b_1, a_1b_2, a_1b_3, \dots, a_mb_n]$.

Finally, the *classification function* $\mathcal{C}$ takes the input $\mathbf{v}$ and output a prediction $\mathbf{y} = \mathcal{C}(\mathbf{v})$.

The overall structure of B-CNN is shown in Fig. 3. A natural candidate for the *feature selection function* f is a CNN consisting of a hierarchy of convolutional and pooling layers. Note that an important design principle of B-CNN is to obtain a low dimension of $f_A(X)$ and $f_B(X)$. Otherwise, the final presentation of the bilinear vector can be very high-dimensional, which makes it impractical for subsequent analysis. To this end, a fully-connected layer with a small number of neurons can be added after the convolutional blocks to further shorten the length of features. Since our model is linear in the output of two CNN streams, we call our architecture bilinear CNN. Intuitively, one CNN stream is expected to extract features of the mask $L(t_1)$, and the other stream to extract features of the masked value $L(t_2)$. Then, the outer product captures the pairwise correlation between the outputs of f_A and f_B , which behaves identically as the product combination of features. Besides, we notice that the *feature selection functions* f_A and f_B can be *independent* or *fully shared* (see Fig. 4). If the computation of f_A and f_B is fully shared, we can save half of the trainable parameters.

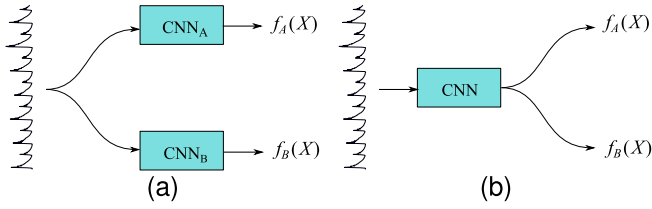


Fig. 4. Feature selection functions (i.e., f_A and f_B) in the B-CNN can (a) be independent, or (b) share all computations.

In experiments, we observe that the two architectures achieve similar attacking performance. Therefore, we adopt the fully shared version of B-CNN in the rest of this paper to reduce the network complexity. Finally, based on the bilinear vector, the *classification function* $\mathcal{C}$ composed of several fully-connected layers completes the network.

In fact, bilinear CNN has already been explored in the deep learning field for fine-grained visual recognition [24] and feature fusion [25]. Inspired by these state-of-the-arts, we adopt B-CNN into profiled SCA for improving second-order profiled attacks. The main advantage of B-CNN is that it overcomes the two limitations of classic CNN models mentioned at the beginning of this section. Recalling that classic CNN models struggle with the tough two-stage learning task (i.e., combination then classification) when performing profiled SCA. Whereas, B-CNN turns the two-stage learning task into a one-stage task by embedding product combination into the network architecture while does not affect the backpropagation of gradients. Unlike second-order DPA that has to perform second-order pre-processing on the raw traces, B-CNN conducts outer product on the low dimensional selected features, which can result in a reasonable dimension of the bilinear vector for efficient classification. On the other side, as the centered product combination function has been proved to be optimal in noisy situations, B-CNN thus can prevent the neural network from falling into a locally optimal solution (i.e., learning a suboptimal combination function) by explicitly leveraging product combination and normalization.

B. Evaluation of Feature Selection and Combination

Because of the black-box nature of neural networks, it can be challenging to interpret their inner-working and decision-making. In this subsection, we propose the layer-wise correlation as a novel tool to evaluate the ability of B-CNN to extract the points of interest (PoIs) and combine the leakage.

1) *Feature Selection*: Recalling that the *feature selection function* is designed to extract features that are relevant to the mask and masked value. Thus, if the B-CNN is perfectly trained, neurons in the last layer of f_A or f_B (i.e., the layer before outer product) must contain information related to the PoIs. For other points that are irrelevant to the classification task, no clear correlation would be detected. Therefore, if we calculate the correlation coefficient between input traces and the output of each neuron in the last layer of f_A and f_B , we can figure out which part of the trace is relevant to the selected features, and thus can reveal the PoIs. More specifically, let us denote $D \in \mathbb{R}^{N_p \times l}$ the input traces, and $E \in \mathbb{R}^{N_p \times q}$ the output matrix of *feature selection function*, where

N_p denotes the number of traces, l is the number of samples per trace, q is the number of features after selection. Thus, by calculating the Pearson correlation coefficients between each column vector of the matrix D and E , we can obtain a correlation matrix $LWC_{sel} \in \mathbb{R}^{q \times l}$, such that the i -th row of the matrix LWC_{sel} is the correlation result between the i -th neuron and each time sample of traces. Specifically, we have:

$$LWC_{sel} = \rho(D, E) = \frac{cov(D, E)}{\sigma_D \sigma_E}. \quad (10)$$

Finally, by visualizing all the row vectors of LWC_{sel} , we can successfully reveal the PoIs by locating the peaks of the correlation curves.

2) *Feature Combination*: To figure out whether and where B-CNN combines the leakage of $L(t_1)$ and $L(t_2)$, we define a LWC_{comb} that calculates the correlation between the HW of intermediate value and the output of each layer of B-CNN. Recalling that neural networks are designed to automatically learn a combination function. Therefore, if a layer of B-CNN has successfully learned the combined features, we could detect high correlations between $HW(Sbox(p_i \oplus k))$ and the output of neurons in this layer. Otherwise, the correlations must be close to zero for all the neurons. Herein, LWC_{comb} is defined as the maximum value of the correlations among all the neurons in a certain layer, i.e.,

$$LWC_{comb} = \max(\rho(HW(Sbox(p_i \oplus k)), E')), \quad (11)$$

where p_i is plaintext, k is the known key, and E' is the output matrix of the layer. We note that not all the layers have a remarkable value of LWC_{comb} . For a shallow layer before the leakages are combined, there is no correlation between the intermediate value and the layer outputs, and the calculated LWC_{comb} is close to zero. For a deep layer where the combination function is successfully learned, we should detect a high correlation value. Therefore, by observing the evaluation of LWC_{comb} depending on the layers of the network, we can figure out whether and where B-CNN learns the combination function. Intuitively, the value of LWC_{comb} could also reveal the goodness of the trained model.

IV. EXPERIMENTS

In this section, we will provide the experimental settings and results on several publicly available datasets. All the experiments are implemented in python using the *PyTorch* library, and are run on a GPU server equipped with 128GB RAM and an NVIDIA GeForce RTX 3090 with 24GB memory. We adopt the *Categorical Cross-Entropy* as the loss function because minimizing cross-entropy has been proved to be equivalent to maximizing the lower bound of mutual information between traces and the secret variable [16]. Optimization is conducted using the *Adam* optimizer and the number of epochs is set to 50. We use the *SeLU* activation function to avoid vanishing and exploding gradient problems. All these hyperparameters are selected following a random search optimization. For each hyperparameter, we select from a predefined finite set of values (see Table I).

TABLE I
HYPERPARAMETERS SEARCH SPACE

	Values
Optimizer	{SGD, Adam, RMSprop}
Activation Function	{ReLU, tanh, ELU, SELU}
Initialization	{He uniform, Uniform distribution}
Pooling	{Average, Maximum}
Epochs	{50, 100, 150, 200}
Batch Size	{50, 100, 200, 600}
Convolution Layers	{1, 2, 3}
Filters	{2, 4, 8, 16, 32, 64, 128}
Kernel Size	{1, 3, 5, 25, 50, 100}
Fully-connected Layers	{1, 2, 3}
Fully-connected Neurons	{5, 10, 15, 20}

A. Datasets

For the experiments, we consider three public datasets that are protected with first-order masking countermeasure and free to access for reproducibility. All these datasets correspond to the implementation of the AES algorithm. In Table II, we summarize the partition of all considered datasets in the experiments.

1) *ASCAD Fixed-Key Dataset*: The ASCAD dataset¹ is introduced by Prouff et al. [15] that targets a software AES implementation running on an 8-bit AVR ATmega8515. The AES state is protected with 16 different masks for the linear parts, while the non-linear Sbox is secured by a pair of input and output masks. Side-channel traces are acquired by recording the electromagnetic radiation emitted by the device with a sampling rate of 2GS/s. This dataset splits into a training set with 50000 training traces and an attack set with 10000 attack traces, each with a uniformly random plaintext and a constant key. Each trace consists of 700 sample points, which contain information on the two pairs of $(Sbox(p[3] \oplus k[3]) \oplus r[3], r[3])$ and $(Sbox(p[3] \oplus k[3]) \oplus r_{out}, r_{out})$ (see Fig. 5a). Besides, each trace is labeled by the expected value of the intermediate value (i.e., the unmasked output of the 3rd Sbox in the first round) such that the label of the i -th trace is:

$$Y(k) = Sbox(p_i[3] \oplus k[3]). \quad (12)$$

Note that although the masks for Sbox are provided, they are not used in the profiling and attacking phases. The fixed-key dataset contains three variants with different desynchronization levels that are referred to as $ASCAD_{sync}^{fixed}$, $ASCAD_{desync50}^{fixed}$, and $ASCAD_{desync100}^{fixed}$. The $ASCAD_{sync}^{fixed}$ dataset corresponds to the original dataset with synchronized traces. For $ASCAD_{desync50}^{fixed}$ and $ASCAD_{desync100}^{fixed}$, each trace is randomly shifted with a maximum window of 50 and 100 samples, respectively.

2) *ASCAD Variable-Key Dataset*: This dataset² is referred to as $ASCAD_{variable}$ that contains 200000 traces for training and 100000 traces for testing. The difference with $ASCAD_{sync}^{fixed}$ is that the training traces have random variable keys and each trace has 1400 features. The 100000 attack

¹This dataset is publicly available at https://github.com/ANSSI-FR/ASCAD/tree/master/ATMEGA_AES_v1/ATM_AES_v1_fixed_key

²This dataset is publicly available at https://github.com/ANSSI-FR/ASCAD/tree/master/ATMEGA_AES_v1/ATM_AES_v1_variable_key

TABLE II
STATISTICAL INFORMATION AND THE PARTITION OF ALL CONSIDERED DATASETS

Dataset	#Feature	#Train	#Validation	#Attack
$ASCAD_{sync}^{fixed}$	700	45000	5000	10000
$ASCAD_{desync50}^{fixed}$	700	45000	5000	10000
$ASCAD_{desync100}^{fixed}$	700	45000	5000	10000
$ASCAD_{variable}^{fixed}$	1400	195000	5000	100000
CHES CTF 2018	2200	43000	1000	5000

TABLE III
NETWORK ARCHITECTURE OF B-CNN ON THE ASCAD FIXED KEY DATASET

Model	Feature selection				Classification	
	filters	kernel size	pool size	fc. layer	bilinear vector	fc. layers
B-CNN $_{sync}^{fixed}$	4	1	2	10	100	10×256
B-CNN $_{desync}^{fixed}$	32	1	2			
	64	50	50	10	100	10×256
	128	3	2			

traces have the same key with random plaintexts and mask values. Each trace is also labeled by the Sbox output in the first round and the leakage model is the same as Equation 12.

3) *CHES CTF 2018 Dataset*: This dataset³ refers to the CHES 2018 Capture-the-flag (CTF) AES-128 trace set. It targets a masked AES-128 implementation running on a 32-bit STM microcontroller. The dataset splits into a training set with 45000 profiling traces and an attack set with 5000 traces. The training set has a fixed key, while the attack set, acquired from a different device, uses a different key. Each trace consists of 2200 features. We note that previous literature only can build HW models such that the label of the i -th trace is:

$$Y(k) = HW(Sbox(p_i[1] \oplus k[1])). \quad (13)$$

While in this paper, we will show that B-CNN can also successfully converge using the identity leakage model $Y(k) = Sbox(p_i[1] \oplus k[1])$, which further confirms the effectiveness of our approach.

B. Results on ASCAD Fixed Key Dataset

For the $ASCAD_{sync}^{fixed}$ dataset, we build the B-CNN model based on a reference CNN model proposed by Zaid et al. [17]. The reference model contains one convolutional block and two fully-connected layers (fc_1 and fc_2), each of which contains 10 neurons. In order to convert the reference model to B-CNN, we preserve its convolutional part and insert a bilinear vector after fc_1 . The bilinear vector combines the leakage of mask and masked value by conducting outer product on the output of fc_1 , and has (10×10) neurons. Then, fc_2 and the output layer complete the network. Since the dimension of this bilinear vector is still very small, this modification has little effect on the network complexity (number of trainable

³This dataset is publicly available at <https://chesctf.riscure.com/2018/content?show=training>

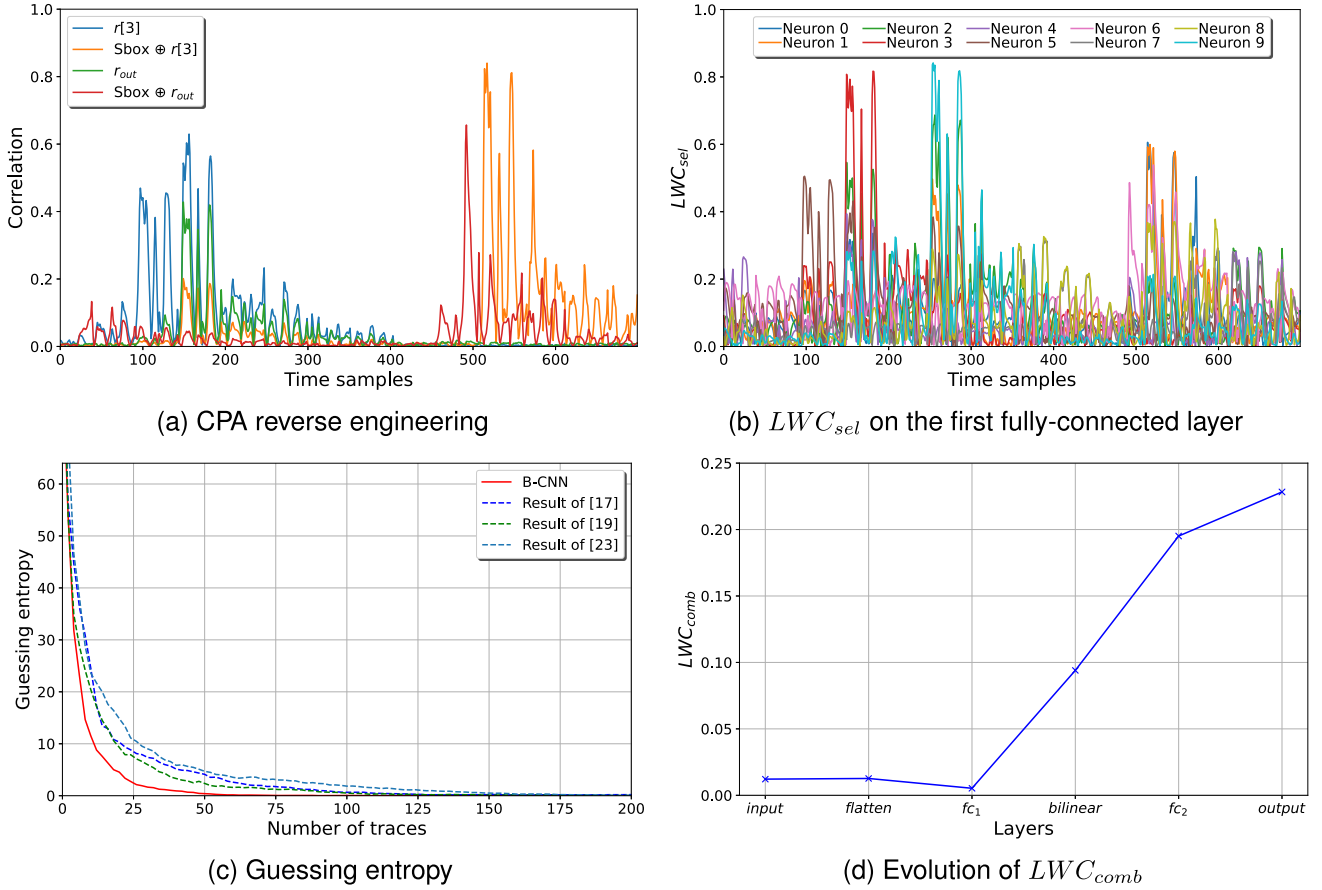


Fig. 5. Attacking results on the $ASCAD_{sync}^{fixed}$ dataset.

parameters). This architecture (see Table III) is referred to as $B-CNN_{sync}^{fixed}$ in the rest of this paper. For this dataset, we use 45000 traces for training, 5000 for the validation, and 10000 for the test. Before the training, traces are preprocessed using feature standardization. The $B-CNN_{sync}^{fixed}$ is trained for 50 epochs with a batch size of 200. During the training phase, we use a *One Cycle Policy* to configure the learning rate in the range $[1 \times 10^{-3}, 1 \times 10^{-2}]$.

Next, we compare our result with the original paper [17] and other state-of-the-art approaches [19], [23]. The results are shown in Fig. 5c. Surprisingly, in terms of Nt_{GE} , $B-CNN_{sync}^{fixed}$ requires two times fewer attack traces (only 95 traces) than the reference models. Meanwhile, the complexity of the network is still within a reasonable range. Further, we evaluate the performance of B-CNN when considering the desynchronization countermeasure. In order to learn shift-invariant features, we modify the feature selection part of the current B-CNN model and use three convolutional layers. The first convolutional layer has a filter number of 32 and then each next layer increases the filter number by a factor of 2. The kernel sizes for the three convolutional layers are 1, 50, and 3, respectively (see Table III). The learning rate is scheduled in the range $[5 \times 10^{-4}, 1 \times 10^{-3}]$ and no pre-processing is conducted. We use the same architecture ($B-CNN_{desync}^{fixed}$) for dataset $ASCAD_{desync50}^{fixed}$ and $ASCAD_{desync100}^{fixed}$. The performances of

TABLE IV
COMPARISON OF PERFORMANCE ON $ASCAD_{sync}^{fixed}$, $ASCAD_{desync50}^{fixed}$ AND $ASCAD_{desync100}^{fixed}$ WITH THE CURRENT STATE-OF-THE-ARTS

Dataset	Metric	[17]	[19]	[23]	B-CNN
$ASCAD_{sync}^{fixed}$	Complexity	16960	6436	79439	17852
	Nt_{GE}	191	≈ 200	202	95
$ASCAD_{desync50}^{fixed}$	Complexity	87279	41052	41321	134186
	Nt_{GE}	244	≈ 250	443	165
$ASCAD_{desync100}^{fixed}$	Complexity	142044	42652	\	134186
	Nt_{GE}	270	≈ 300	\	177

B-CNNs and reference models are summarized in Table IV. Although desynchronization makes it difficult for the network to converge, we can successfully recover the key using only about 170 traces, which outperforms the current state-of-the-art results. A more fine-grained hyperparameter tuning may further boost the performance of B-CNN, which is out of the scope of this paper.

Further, we consider a more challenging attack scenario when the attacker only has a limited number of profiling traces. Herein, we conduct more experiments on the $ASCAD_{sync}^{fixed}$, $ASCAD_{desync50}^{fixed}$, and $ASCAD_{desync100}^{fixed}$ by varying the profiling traces in the range [8000, 40000]. We compare the performance of B-CNN with the reference models [17], [19],

TABLE V
EVOLUTION OF Nt_{GE} DEPENDING ON THE NUMBER OF PROFILING TRACES N_p

Nt_{GE}		N_p	8000	10000	15000	20000	25000	30000	35000	40000
ASCAD ^{fixed} _{sync}	[17]		>10000	>10000	3170	1193	543	476	358	352
	[19]		>10000	>10000	1835	1129	515	313	302	268
	[23]		>10000	>10000	3352	1218	801	655	466	333
	B-CNN		1882	1078	633	473	277	238	212	168
ASCAD ^{fixed} _{desync50}	[17]		>10000	>10000	4762	1369	921	611	593	442
	[19]		>10000	>10000	5650	1538	876	744	529	435
	[23]		>10000	>10000	>10000	>10000	>10000	2584	1264	592
	B-CNN		2468	1350	883	537	499	374	281	230
ASCAD ^{fixed} _{desync100}	[17]		>10000	>10000	8692	3488	1427	914	658	541
	[19]		>10000	>10000	9180	5842	2093	1226	962	725
	B-CNN		5500	5478	1147	706	623	459	381	277

Nt_{GE} is averaged over 10 trained models.

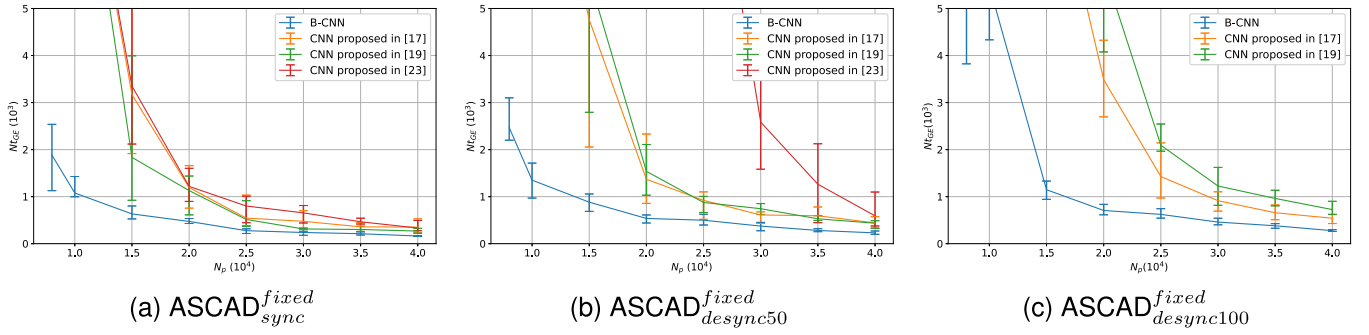


Fig. 6. Evolution of Nt_{GE} depending on the number of profiling traces N_p (ASCAD dataset).

[23]. In order to make the comparison as fair as possible, we rerun all these networks in the same setting (i.e., same learning rate, batch size, and training epochs). When rerunning the reference models, we notice that training the models as proposed in [17] and [23] does not always result in a converging model (similar results were reported in [19]). Therefore, we only report the Nt_{GE} of the converged models. The attacking results are summarized in Fig. 6 and Table V. Surprisingly, B-CNN can consistently converge and requires significantly fewer traces to reach a GE of 0 even using a limited number of profiling traces. For example, when the model is trained with the B-CNN structure on the ASCAD^{fixed}_{sync} dataset, only 8000 profiling traces are needed to perform a successful attack while more than 15000 traces are needed to reach the same performance when using the architecture proposed in [17]. As far as we know, we are the first to report successful profiled attacks using only 8000 profiling traces on the ASCAD dataset. The results confirm that B-CNN could make it easier and faster for the network to converge.

Recalling that B-CNN uses a *feature selection function* to extract features related to mask and masked value. To validate our results, we first use the knowledge of the key and the masks to perform a reverse engineer CPA to highlight the locations of the mask and masked value (see Fig. 5a). Then we calculate the LWC_{sel} on the first fully-connected layer that contains 10 neurons. As is shown in Fig. 5b, LWC_{sel} reveals two main areas, i.e., [100, 400] and [500, 700], which match with the areas of the mask and masked value obtained through reverse engineering CPA. The results confirm that the layer

before the product combination has successfully extracted the PoIs and 10 neurons are enough for feature selection. Note that the locations highlighted by LWC_{sel} are obtained without any knowledge of the mask and the key.

To further validate that B-CNN has indeed combined the leakage exactly in the bilinear layer, we calculate the LWC_{comb} on each layer of B-CNN. As is shown in Fig. 5d, the LWC_{comb} on the input layer is about zero, which is not surprising because there is no second-order leakage for the ASCAD dataset. For the flatten layer and fc_1 , the LWC_{comb} is also close to zero because the *feature selection function* is designed only to extract features related to mask and masked value, rather than to combine them. As we expect, LWC_{comb} detects a high correlation on the bilinear vector, which confirms our assumption. The values of LWC_{comb} on fc_2 and output layer are higher because they further combine the leakage of multiple neurons. In fact, the outputs of the last three layers can be directly exploited by first-order correlation power analysis (CPA), which is out of the scope of this paper.

C. Results on ASCAD Random Key Dataset

In contrast to the previous datasets, ASCAD^{variable} has random keys for profiling, and has 1400 features per trace. For this dataset, we use a similar network architecture as the one used in ASCAD^{fixed}_{sync} (with a slight difference in the number of filters and neurons). Table VI shows the detailed architecture of B-CNN^{variable}. Besides, we use 195000 profiling traces for training, 5000 profiling traces for validation,

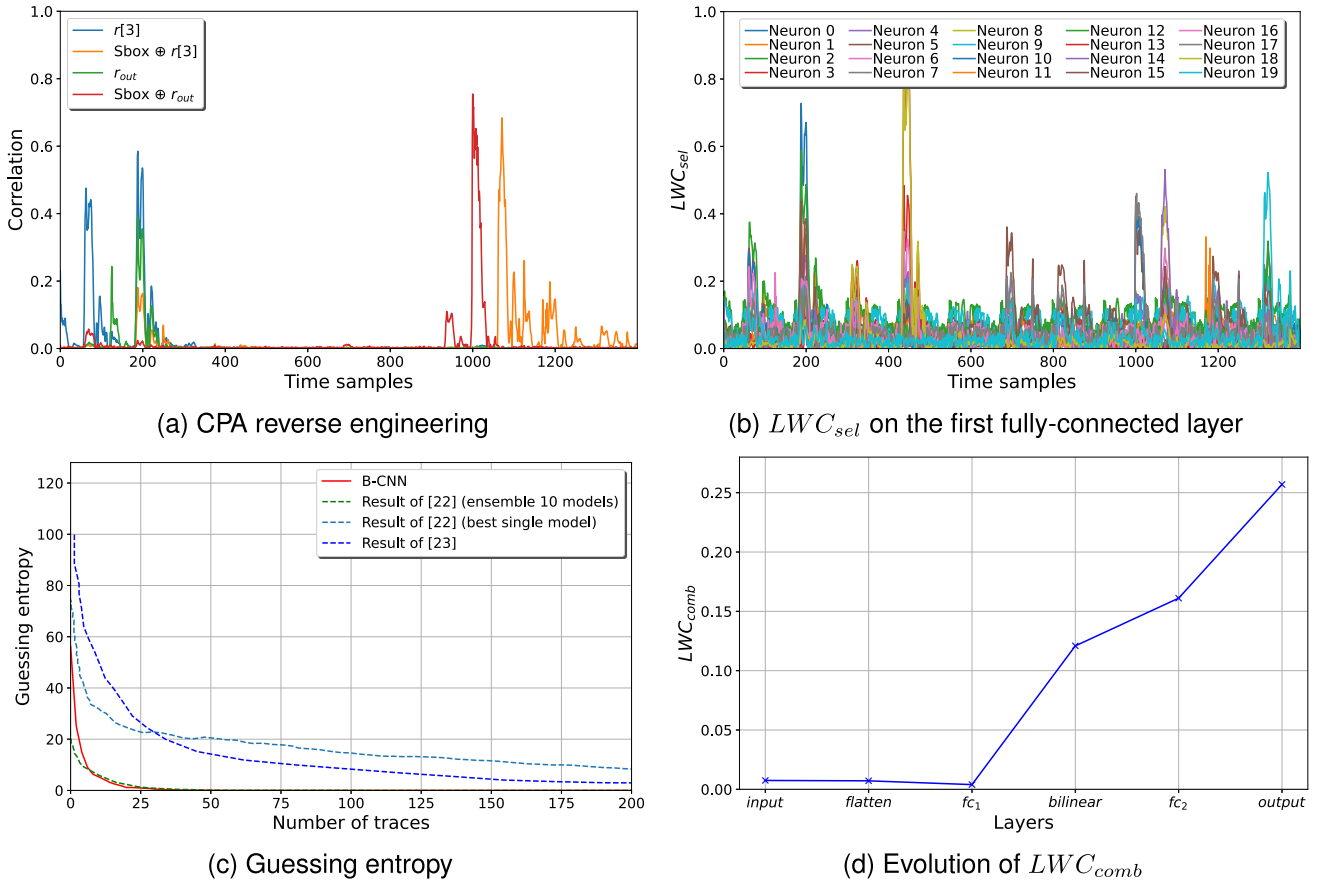
Fig. 7. Attacking results on the $ASCAD^{variable}$ dataset.

TABLE VI
NETWORK ARCHITECTURE OF B-CNN ON THE ASCAD
VARIABLE KEY DATASET

Model	Feature selection				Classification	
	filters	kernel size	pool size	fc. layer	bilinear vector	fc. layers
B-CNN ^{variable}	2	1	2	20	400	20 × 256

TABLE VII
COMPARISON OF PERFORMANCE ON $ASCAD^{variable}$ WITH THE CURRENT
STATE-OF-THE-ARTS

Dataset	Metric	[22]	[23]	[30]	B-CNN
$ASCAD_{sync}^{variable}$	Complexity	N/A	70492	2076744	41424
	Nt_{GE}	≈ 105	490	1568	57

and 100000 attack traces for the test. All the traces are preprocessed with feature standardization. Thanks to the one cycle policy, we are able to configure our learning rate in the range $[5 \times 10^{-4}, 5 \times 10^{-3}]$. The network is trained for 50 epochs with a batch size of 600.

Fig. 7c shows the result of GE on the $ASCAD^{variable}$ dataset. We see that the key can be stably recovered within 60 attack traces. Next, we compare our result with the current state-of-the-art in Table VII. It is worth mentioning that although [23] and [30] conduct careful hyperparameter tuning using traditional CNN architecture, the proposed B-CNN^{variable} requires significantly fewer traces for GE to reach 0. Beside, although direct comparison with [22] is not possible (because the author use ensembles), B-CNN^{variable} still outperforms [22] that use an ensemble of 10 models. Finally, in terms of complexity, we notice that our network is also significantly less complex than the reference papers.

By visualizing the LWC_{sel} (see Fig. 7b), we can evaluate that the B-CNN^{variable} successfully recognizes relevant information related to the mask and masked value, although several sensitive areas (between samples 400 and 500) is detected by our network whereas nothing appears on the results of CPA reverse engineering. Fig. 7d shows the result of LWC_{comb} on each layer of B-CNN^{variable}. Like the ASCAD fixed key dataset, LWC_{comb} only detects high correlations after the bilinear combination, which confirms our previous result.

D. Results on CHES CTF 2018 Dataset

Unlike the previous ASCAD dataset, the profiling traces and attack traces of the CHES CTF dataset are acquired from two different devices. Besides, the traces are much longer (i.e., 2200 samples per trace). We notice that all previous works only report their results on the CHES CTF dataset with the HW leakage model, because their networks cannot generalize well or converge with the identity leakage model. Herein, we show

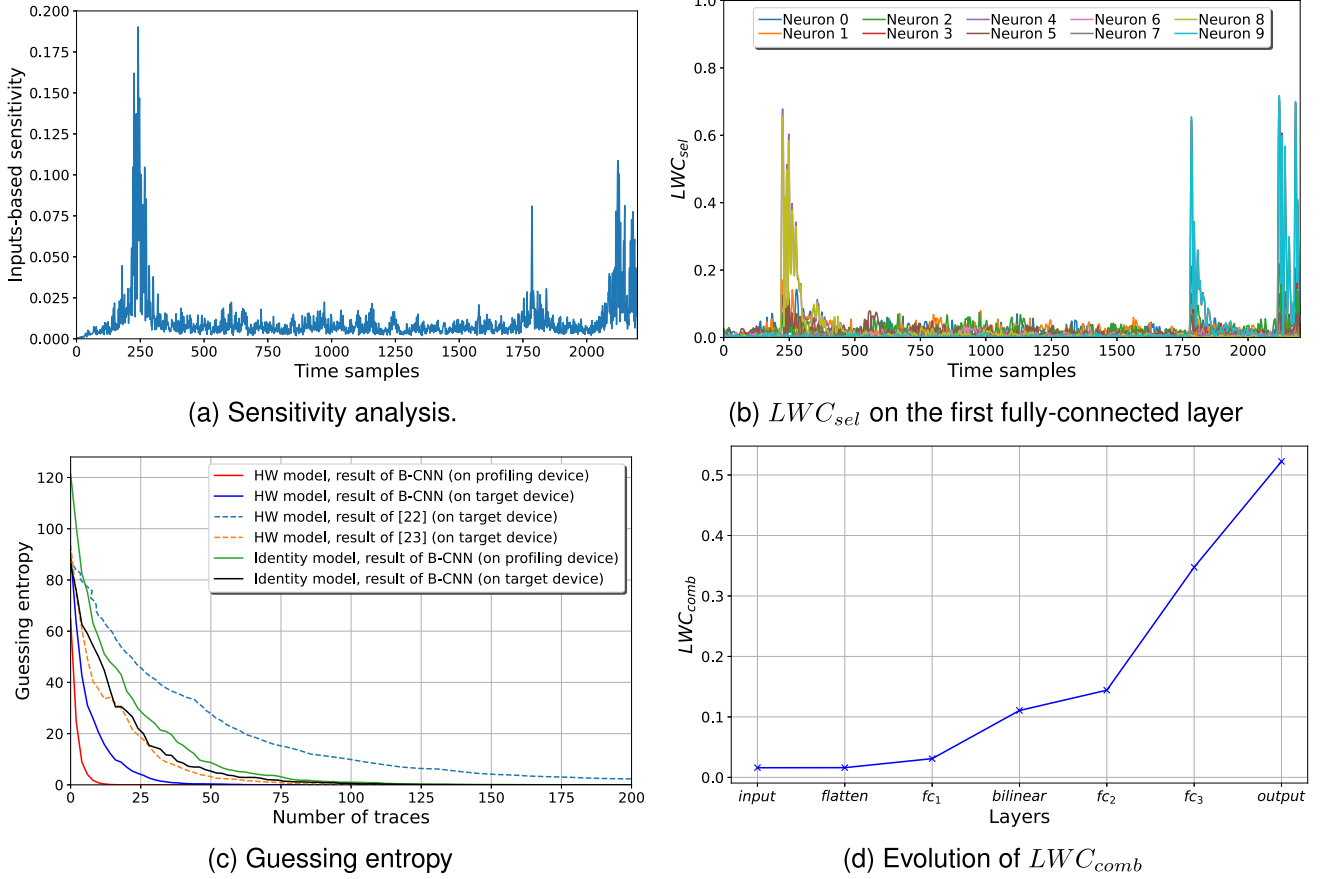


Fig. 8. Attacking results on the CHES CTF 2018 dataset.

TABLE VIII

NETWORK ARCHITECTURE OF B-CNN ON THE CHES CTF 2018 DATASET

Leakage model	Feature selection				Classification	
	filters	kernel size	pool size	fc. layer	bilinear vector	fc. layers
HW	4	100	4	10	100	$10 \times 10 \times 9$
identity	2	1	2	10	100	$20 \times 20 \times 256$

that using B-CNN, we can successfully recover the key of the target device with both the HW and identity leakage models.

For the HW leakage model, we build B-CNN^{HW} based on the architecture reported in [23]. We use 43000 traces for training, 1000 traces for validation, and 5000 traces for the test. Since profiling traces and attack traces are acquired from different devices, we perform feature standardization separately on the two sets of traces to prevent the influence of DC offset [31]. As before, the learning rate is configured to vary in the range $[5 \times 10^{-4}, 5 \times 10^{-3}]$. We train the network for 50 epochs with a batch size of 200. For the identity leakage model, we use a similar architecture as the one used in the ASCAD^{variable} dataset. The detailed structure of network is concluded in Table VIII. When training the identity model, we adopt *dropout* [32] with a drop rate of 0.25 to prevent overfitting, which makes the model converge stably.

TABLE IX

COMPARISON OF PERFORMANCE ON THE CHES CTF 2018 DATASET WITH THE CURRENT STATE-OF-THE-ARTS

Leakage model	Metric	[22]	[23]	[30]	B-CNN
HW	Complexity	N/A	33788	2418085	22633
	Nt_{GE}	310	122	618	65
Identity	Complexity	\	\	\	29834
	Nt_{GE}	\	\	\	182

Fig. 8c shows the results of B-CNN models on both the profiling device and the target device. When using the HW leakage model, we can successfully recover the key of the profiling device within 20 traces. Attacking the target device requires more traces (about 65 traces), which is not surprising because the device discrepancy usually results in two different distributions of leakage [33]. The results are slightly worse when using the identity leakage model. Specifically, we need about 180 traces to recover the key of the profiling or target device. However, as far as we know, we are the first to report successful profiled attacks using the identity leakage model on the CHES CTF 2018 dataset.

Next, we compare our results with previous literature in Table IX. As we can see, B-CNN requires significantly fewer attack traces to recover the key. In terms of network complexity, our architecture also has the smallest number of trainable parameters.

As before, we show the result of LWC_{sel} in Fig. 8b. Since this dataset does not release the mask-related information, we cannot perform the reverse engineering CPA to highlight the position of the mask and masked value. Therefore, we decide to compare the result of LWC_{sel} with the sensitivity analysis [34]. As we expect, the result of sensitivity analysis (see Fig. 8a) is highly consistent with our approach, which confirms that B-CNN can successfully extract the PoIs related to the mask and masked value. The result of LWC_{comb} further confirms that B-CNN has combined the leakage after the bilinear layer.

V. DISCUSSIONS

A. How Would B-CNN Scale for More Than Two Shares?

Although first-order masking is the most widely used in the SCA community, there are also masking schemes that have more than two shares [9]. Therefore, it is meaningful to figure out how our approach can be generalized in the presence of higher-order masking.

In this work, the proposed B-CNN is designed for second-order analysis. However, such an architecture can be naturally extended to target higher-order masking. For example, when there are three shares, we can convert the bilinear to a trilinear model (called T-CNN) by introducing another CNN stream. Herein, we acquire the power traces from a device that we fully control (an 8-bit AVR microcontroller, ATmega128A). To control the leakage better, we do not actually implement a masked AES, but simulate the leakages of different shares by an input array (with 3 bytes) through the assembly instructions like LD and ST (similar techniques were used in [16] and [35]). Specifically, we use the simulated output of Sbox as the label (i.e., the XORed value of the three input bytes, which simulate the m_1, m_2 and $Sbox(p \oplus k) \oplus m_1 \oplus m_2$). Traces are collected by a Pico 3203D oscilloscope at a sampling rate of 125MS/s while the chip runs at 11MHz. We acquire 200000 traces, while each trace contains 1200 time samples.

In our experiments, both the T-CNN and B-CNN are constructed based on a classic CNN that has three convolutional layers and three fully-connected layers. The first convolutional layer has a filter number of 4 and then each next layer increases the filter number by a factor of 2. The kernel sizes for the three convolutional layers are 1, 11, and 3, respectively. The numbers of neurons for the fully-connected layers are {10, 50, 20}. As before, we first convert the classic CNN to a B-CNN by inserting a bilinear vector after the first fully-connected layer. Since there are three shares to be combined, we further convert the B-CNN to a trilinear model (T-CNN). The only difference between the B-CNN and T-CNN is that the bilinear vector is replaced by a trilinear one that is the product output of three CNN streams. We use 190000 traces for training, 5000 traces for validation, and 5000 traces for the test. Networks are trained for 50 epochs with a batch size of 200. We use the Adam optimizer and the learning rate is configured to vary in the range $[2 \times 10^{-4}, 2 \times 10^{-3}]$.

To evaluate the feature selection ability of T-CNN, we first use the knowledge of the input bytes to perform a reverse engineer CPA to highlight the location of the three shares (see

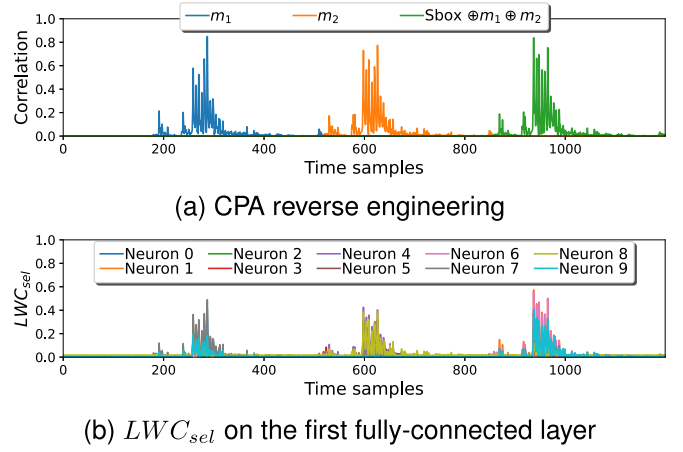


Fig. 9. Evaluation of feature selection capability of T-CNN.

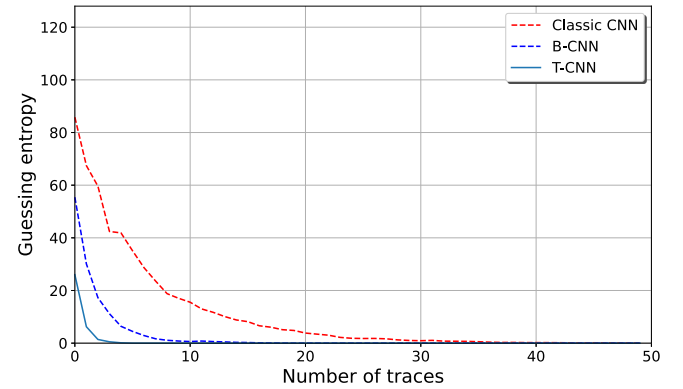


Fig. 10. Comparison of the attacking performance between classic CNN and our architectures.

Fig. 9a). Then, we calculate the LWC_{sel} on the first fully-connected layer. As is shown in Fig. 9b, LWC_{sel} reveals three areas, which exactly match the areas of the three shares. Next, we compare the performance of T-CNN, B-CNN, and classic CNN in Fig. 10. We can observe that B-CNN still works for an implementation that has three shares and could outperform the classic CNN. Besides, T-CNN achieves the best performance and could successfully recover the key with only 6 traces. We can conclude from the results that our approach can be naturally generalized in the higher-order masking scenario.

B. What About Other Masking Schemes?

Besides Boolean masking, there are other masking schemes such as multiplicative masking [36] and affine masking [37]. Although the effectiveness of the centered product combining has been only studied in [13], [26] in the context of Boolean masking, Fumaroli et al. showed that this combining function stays a natural choice against any kind of masking [38]. Since the effectiveness of B-CNN is based on the embedding of product combining in CNN, we would argue that B-CNN is also suitable for other masking schemes. As far as we know, Boolean masking is nowadays the most widely used. Given that higher-order countermeasures other than Boolean do not

appear to be ready for practical applications, we leave the evaluation of B-CNN on other masking schemes for future work.

VI. CONCLUSION

In this paper, we introduce and analyze the interpretability of bilinear convolutional neural networks for second-order profiled SCA. We demonstrate that B-CNN is easier to converge since it simplifies the learning task of neural networks. Besides, B-CNN could prevent the network from falling into a locally optimal solution by explicitly embedding the product combination. To evaluate our methodology, we propose a visualization tool called layer-wise correlation (i.e., LWC_{sel} and LWC_{comb}). We show that LWC_{sel} can help analyze which part of the traces are most relevant during the training process, while LWC_{comb} can reveal when the network really combines the leakage and help to evaluate the effectiveness of the combination. We test our methodology on three public datasets that cover a wide range of use-cases. In all cases, our methodology outperforms the current state-of-the-art. Specifically, our attacks with B-CNN are much faster and the architectures are also less complex. We note that this paper is not focused on hyperparameter tuning and our methodology could benefit from techniques such as data augmentation and ensembles.

REFERENCES

- [1] P. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," in *Proc. Annu. Int. Cryptol. Conf.* Berlin, Germany: Springer, 1999, pp. 388–397.
- [2] D. Agrawal, B. Archambeault, J. R. Rao, and P. Rohatgi, "The EM side—Channel (s)," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.* Berlin, Germany: Springer, 2002, pp. 29–45.
- [3] S. Chari, J. R. Rao, and P. Rohatgi, "Template attacks," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.* Berlin, Germany: Springer, 2002, pp. 13–28.
- [4] G. Hospodar, B. Gierlichs, E. De Mulder, I. Verbauwhede, and J. Vandewalle, "Machine learning in side-channel analysis: A first study," *J. Cryptograph. Eng.*, vol. 1, no. 4, pp. 293–302, Dec. 2011.
- [5] A. Heuser and M. Zohner, "Intelligent machine homicide," in *Proc. Int. Workshop Constructive Side-Channel Anal. Secure Design.* Berlin, Germany: Springer, 2012, pp. 249–264.
- [6] T. Bartkewitz and K. Lemke-Rust, "Efficient template attacks based on probabilistic multi-class support vector machines," in *Proc. Int. Conf. Smart Card Res. Adv. Appl.* Berlin, Germany: Springer, 2012, pp. 263–276.
- [7] L. Lerman, G. Bontempi, and O. Markowitch, "Power analysis attack: An approach based on machine learning," *Int. J. Appl. Cryptogr.*, vol. 3, no. 2, pp. 97–115, 2014.
- [8] S. Chari, C. S. Jutla, J. R. Rao, and P. Rohatgi, "Towards sound approaches to counteract power-analysis attacks," in *Proc. Annu. Int. Cryptol. Conf.* Berlin, Germany: Springer, 1999, pp. 398–412.
- [9] M. Rivain and E. Prouff, "Provably secure higher-order masking of AES," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.* Berlin, Germany: Springer, 2010, pp. 413–427.
- [10] J.-S. Coron, "Higher order masking of look-up tables," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn.* Berlin, Germany: Springer, 2014, pp. 441–458.
- [11] K. Schramm and C. Paar, "Higher order masking of the AES," in *Proc. Cryptograph. Track RSA Conf.* Berlin, Germany: Springer, 2006, pp. 208–225.
- [12] T. S. Messerges, "Using second-order power analysis to attack DPA resistant software," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.* Berlin, Germany: Springer, 2000, pp. 238–251.
- [13] A. A. Ding, L. Zhang, Y. Fei, and P. Luo, "A statistical model for higher order DPA on masked devices," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.* Berlin, Germany: Springer, 2014, pp. 147–169.
- [14] H. Maghrebi, T. Portigliatti, and E. Prouff, "Breaking cryptographic implementations using deep learning techniques," in *Proc. Int. Conf. Secur., Privacy, Appl. Cryptogr. Eng.* Berlin, Germany: Springer, 2016, pp. 3–26.
- [15] E. Prouff, R. Strullu, R. Benadjila, E. Cagli, and C. Dumas, "Study of deep learning techniques for side-channel analysis and introduction to ASCAD database," *Cryptol. ePrint Arch.*, Paper 2018/053, 2018. [Online]. Available: <https://eprint.iacr.org/2018/053>, doi: 10.1007/s13389-019-00220-8.
- [16] L. Masure, C. Dumas, and E. Prouff, "A comprehensive study of deep learning for side-channel analysis," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2020, pp. 348–375, Nov. 2020.
- [17] G. Zaid, L. Bossuet, A. Habrard, and A. Venelli, "Methodology for efficient CNN architectures in profiling attacks," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2020, no. 1, pp. 1–36, 2020.
- [18] E. Cagli, C. Dumas, and E. Prouff, "Convolutional neural networks with data augmentation against jitter-based countermeasures," in *Proc. Int. Conf. Cryptograph. Hardw. Embedded Syst.* Berlin, Germany: Springer, 2017, pp. 45–68.
- [19] L. Wouters, V. Arribas, B. Gierlichs, and B. Preneel, "Revisiting a methodology for efficient CNN architectures in profiling attacks," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2020, pp. 147–168, Jun. 2020.
- [20] Y.-S. Won, X. Hou, D. Jap, J. Breier, and S. Bhasin, "Back to the basics: Seamless integration of side-channel pre-processing in deep neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3215–3227, 2021.
- [21] U. Rioja, L. Batina, I. Armendariz, and J. L. Flores, "Towards human dependency elimination: AI approach to SCA robustness assessment," *Cryptol. ePrint Archive*, p. 1316, Sep. 2021.
- [22] G. Perin, Ł. Chmielewski, and S. Picek, "Strength in numbers: Improving generalization with ensembles in machine learning-based profiled side-channel analysis," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2020, pp. 337–364, Aug. 2020.
- [23] J. Rijdsdijk, L. Wu, G. Perin, and S. Picek, "Reinforcement learning for hyperparameter tuning in deep learning-based side-channel analysis," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2021, pp. 677–707, Jul. 2021.
- [24] T.-Y. Lin, A. RoyChowdhury, and S. Maji, "Bilinear CNN models for fine-grained visual recognition," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1449–1457.
- [25] A. Fukui, D. H. Park, D. Yang, A. Rohrbach, T. Darrell, and M. Rohrbach, "Multimodal compact bilinear pooling for visual question answering and visual grounding," 2016, *arXiv:1606.01847*.
- [26] E. Prouff, M. Rivain, and R. Bevan, "Statistical analysis of second order differential power analysis," *IEEE Trans. Comput.*, vol. 58, no. 6, pp. 799–811, Jan. 2009.
- [27] E. Oswald and S. Mangard, "Template attacks on masking—Resistance is futile," in *Proc. Cryptograph. Track RSA Conf.* Berlin, Germany: Springer, 2007, pp. 243–256.
- [28] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.
- [29] S. Picek, A. Heuser, A. Jovic, S. Bhasin, and F. Regazzoni, "The curse of class imbalance and conflicting metrics with machine learning for side-channel evaluations," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2019, no. 1, pp. 1–29, 2019.
- [30] L. Wu, G. Perin, and S. Picek, "I choose you: Automated hyperparameter tuning for deep learning-based side-channel analysis," *Cryptol. ePrint Arch.*, p. 1293, Oct. 2020.
- [31] D. P. Montminy, R. O. Baldwin, M. A. Temple, and E. D. Laspe, "Improving cross-device attacks using zero-mean unit-variance normalization," *J. Cryptogr. Eng.*, vol. 3, no. 2, pp. 99–110, 2013.
- [32] N. S. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2019.
- [33] M. Renaud, F.-X. Standaert, N. Veyrat-Charvillon, D. Kamel, and D. Flandre, "A formal study of power variability issues and side-channel attacks for nanoscale devices," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn.* Berlin, Germany: Springer, 2011, pp. 109–128.
- [34] B. Timon, "Non-profiled deep learning-based side-channel attacks with sensitivity analysis," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2019, pp. 107–131, Feb. 2019.
- [35] X. Lu, C. Zhang, P. Cao, D. Gu, and H. Lu, "Pay attention to raw traces: A deep learning architecture for end-to-end profiling attacks," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2021, pp. 235–274, Jul. 2021.

- [36] M.-L. Akkar and C. Giraud, "An implementation of DES and AES, secure against some attacks," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.* Berlin, Germany: Springer, 2001, pp. 309–318.
- [37] M. V. Willich, "A technique with an information-theoretic basis for protecting secret data from differential power attacks," in *Proc. IMA Int. Conf. Cryptogr. Coding*. Berlin, Germany: Springer, 2001, pp. 44–62.
- [38] G. Fumaroli, A. Martinelli, E. Prouff, and M. Rivain, "Affine masking against higher-order side channel analysis," in *Proc. Int. Workshop Sel. Areas Cryptogr.* Berlin, Germany: Springer, 2010, pp. 262–280.



Pei Cao received the B.S. degree in information security from the University of Electronic Science and Technology of China, Chengdu, in 2018. He is currently pursuing the Ph.D. degree with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai. His research interests include side-channel analysis, hardware security, and deep learning.



Chi Zhang received the B.S. degree in computer science and technology from Southeast University, Nanjing, in 2014. He is currently pursuing the Ph.D. degree with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai. His research interests include hardware security, signal processing, and machine learning for side-channel analysis.



Xiangjun Lu received the B.S. degree in software engineering from Northwestern Polytechnical University, Xi'an, in 2015. He is currently pursuing the Ph.D. degree with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai. His research interests include side-channel analysis, hardware security, machine learning, and neural networks.



Dawu Gu received the B.S. degree in applied mathematics from Xidian University, Xi'an, China, in 1992, and the M.S. and Ph.D. degrees in cryptography in 1995 and 1998, respectively. He is currently a Distinguished Professor at the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University (SJTU), Shanghai, where he leads the Laboratory of Cryptology and Computer Security (LoCCS). He has got over 150 scientific papers in academic journals and conferences and owned 28 innovation patents. His research interests include crypto algorithms, crypto engineering, and system security. He was the Winner of Chang Jiang Scholars Distinguished Professors Program in 2014 by Ministry of Education of China. He won the National Award of Science and Technology Progress in 2017.



Sen Xu received the Ph.D. degree in computer science from Shanghai Jiao Tong University, Shanghai, in 2018. He is currently working at View-Sources Technology Company Ltd., Shanghai. His research interests include side-channel analysis, leakage resilient, cryptographic implementations, and hardware security.